

PW EARNERS
DSA in C++

Daily Practice Problems (DPP) — 6

Topic: Arrays and Vectors in C++

Q1. State **True** or **False** for each statement below regarding Arrays and Vectors.

- (i) The size of a static array in C++ can be dynamically changed during runtime
after its declaration.
- (ii) Attempting to access `arr[-1]` in C++ will always trigger a direct compilation error.
- (iii) The expression `sizeof(arr)/sizeof(arr[0])` accurately calculates the
number of elements in a static array.
- (iv) When an array is passed to a function using `void solve(int arr[])`, it is
fundamentally passed by reference (as a pointer).
- (v) The statement `vector<int> v(5, 10);` initializes a vector of size 5, where
every element holds the value 10.
- (vi) The `capacity()` of a vector in C++ is always strictly equal to its `size()`.
- (vii) Elements of a standard C++ array are allocated in contiguous memory locations.

Multiple Choice Questions (Theoretical Understanding)

Q2. Which of the following is the correct syntax to declare a dynamic array (vector) of size N , with all elements initialized to zero?

- | | |
|---|----------------------------------|
| A. <code>vector<int> v(N, 0);</code> | C. <code>vector v(N) = 0;</code> |
| B. <code>vector<int> v[N] = {0};</code> | D. <code>int v[N] = {0};</code> |

Q3. What internally happens when a `std::vector` capacity is full and a new element is added using `push_back()`?

- | | |
|--|---|
| A. The program throws an overflow error. | block of memory (usually double the size) and moves elements. |
| B. The program simply overwrites the last element. | |
| C. The vector automatically allocates a new | D. The capacity strictly increases by exactly 1 element. |

Q4. Which function is used to remove the last element of a vector?

- A. `erase_back()` B. `pop_back()` C. `remove()` D. `delete_back()`

Q5. What is the amortized average time complexity of pushing an element to the back of a vector?

- A. $O(1)$ B. $O(N)$ C. $O(\log N)$ D. $O(N^2)$

Q6. Consider the declaration `int arr[5];`. What does the bare identifier `arr` fundamentally represent in expressions?

- A. The value of the first element. C. The total memory size of the array.
B. The memory address of the first element. D. A reference to the entire array block.

Q7. How should you correctly pass a vector to a function if you want to modify the original vector without copying it?

- A. `void modify(vector<int> v)` C. `void modify(vector<int>[] v)`
B. `void modify(vector<int>& v)` D. `void modify(ref vector<int> v)`

For Q8 to Q18: Write the exact output of each program snippet. Assume all required headers like `<iostream>` and `<vector>` are included, and `using namespace std;` is present. If there is a compilation error, undefined behavior, or garbage value, state it clearly and explain briefly.

Q8.

```
1 int main(){
2     int arr[4] = {10, 20};
3     cout << arr[1] << " " << arr[3];
4 }
```

Output:

.....

Q9.

```
1 int main(){
2     vector<int> v;
3     v.push_back(5);
4     v.push_back(10);
5     v.push_back(15);
6     v.pop_back();
7     cout << v.size() << " " << v.capacity();
8 }
```

Output:

.....

Q10.

```
1 void updateVector(vector<int> v) {
2     v[0] = 100;
```

```
3 }
4 int main(){
5     vector<int> vec = {1, 2, 3};
6     updateVector(vec);
7     cout << vec[0];
8 }
```

Output:

.....

Q11.

```
1 void updateArray(int arr[]) {
2     arr[0] = 100;
3 }
4 int main(){
5     int arr[] = {1, 2, 3};
6     updateArray(arr);
7     cout << arr[0];
8 }
```

Output:

.....

Q12.

```
1 int main(){
2     int arr[3];
3     arr[0] = 1; arr[1] = 2; arr[2] = 3;
4     cout << arr[4];
5 }
```

Output:

.....

Q13.

```
1 int main(){
2     vector<int> v(3, 5);
3     v.push_back(10);
4     cout << v[2] << " " << v[3];
5 }
```

Output:

.....

Q14.

```
1 int main(){
2     int arr[] = {10, 20, 30};
3     cout << *(arr + 1) << " " << arr[2];
4 }
```

Output:

Q15.

```
1 int main(){
2     int arr[5] = {1, 2};
3     cout << sizeof(arr) / sizeof(arr[0]) << " " << arr[4];
4 }
```

Output:

Q16.

```
1 int main(){
2     vector<int> v = {1, 2, 3};
3     for(int &x : v) {
4         x = x * 2;
5     }
6     cout << v[1];
7 }
```

Output:

Q17. Crazy Question

```
1 int main(){
2     int arr[] = {2, 4, 6, 8};
3     int* ptr = arr;
4     cout << *ptr++ << " ";
5     cout << ++*ptr << " ";
6     cout << ++*ptr << endl;
7 }
```

Output:

Q18. Crazy Question

```
1 int main(){
2     vector<int> v = {10, 20, 30};
3     int& ref = v[0];
4
5     // Force vector to grow and reallocate
6     for(int i = 0; i < 100; i++) {
7         v.push_back(i);
8     }
9
10    ref = 999;
11    cout << v[0] << endl;
12 }
```

Output:

For Q19 to Q30: Write clean, optimal, and compilable C++ programs. Assume inputs are provided by the user. Do NOT use predefined STL algorithms like `sort()` or `reverse()` unless specified.

Q19. Find the **Third Maximum** element in a given array.

Constraints: $3 \leq N \leq 10^5$, $-10^9 \leq arr[i] \leq 10^9$. You are **not** allowed to sort the array.

Sample Input: $N = 5$, $arr = \{4, 1, 9, 7, 2\} \Rightarrow$ *Sample Output:* 4

Explanation: The max is 9, second max is 7, third max is 4.

Q20. Complex Array Rotation. Given an array of size N , first right-rotate the array by X positions, and then left-rotate the resulting array by Y positions. Write an optimal $O(N)$ approach.

Constraints: $1 \leq N \leq 10^5$, $0 \leq X, Y \leq 10^9$, $-10^9 \leq arr[i] \leq 10^9$.

Sample Input: $N = 5$, $arr = \{1, 2, 3, 4, 5\}$, $X = 2$, $Y = 1$

Sample Output: $\{5, 1, 2, 3, 4\}$

Explanation: Right by 2 gives $\{4, 5, 1, 2, 3\}$. Then left by 1 gives $\{5, 1, 2, 3, 4\}$.

Q21. Check if an array is strictly sorted in **non-increasing** order. Return **true** if every element is less than or equal to the previous element, else **false**.

Constraints: $1 \leq N \leq 10^5$, $-10^9 \leq arr[i] \leq 10^9$.

Sample Input 1: $\{8, 5, 5, 2\} \Rightarrow$ *Output:* true

Sample Input 2: $\{4, 2, 8\} \Rightarrow$ *Output:* false

Q22. Given an array and a target integer T , count how many elements in the array are **strictly less** than T .

Constraints: $1 \leq N \leq 10^5$, $-10^9 \leq arr[i], T \leq 10^9$.

Sample Input: $arr = \{5, 2, 8, 1, 9\}$, $T = 6 \Rightarrow$ *Output:* 3 (Elements 5, 2, 1)

Q23. First Repeated Element. Given an array of size N , find the first element from the left that appears more than once in the array. If no element repeats, print -1.

Constraints: $1 \leq N \leq 10^5$, $0 \leq arr[i] \leq 10^5$.

Sample Input: $N = 7$, $arr = \{10, 5, 3, 4, 3, 5, 6\} \Rightarrow$ *Output:* 5

Explanation: Both 5 and 3 are repeated, but traversing from the left, 5 is the first element we encounter that will be repeated later.

Q24. Move all 0s to the end of the array while maintaining the relative order of the non-zero elements.

Constraints: $1 \leq N \leq 10^5$, $0 \leq arr[i] \leq 10^9$.

Sample Input: $arr = \{0, 1, 0, 3, 12\} \Rightarrow$ *Output:* $\{1, 3, 12, 0, 0\}$

Q25. Check if an array is a **Palindrome**. An array is a palindrome if it reads the same forward and backward.

Constraints: $1 \leq N \leq 10^5$, $-10^9 \leq arr[i] \leq 10^9$.

Sample Input 1: $\{1, 2, 3, 2, 1\} \Rightarrow$ *Output:* Palindrome

Sample Input 2: $\{1, 2, 3, 4\} \Rightarrow$ *Output:* Not Palindrome

Q26. Count the number of **Even** and **Odd** numbers in a given array.

Constraints: $1 \leq N \leq 10^5$, $0 \leq arr[i] \leq 10^9$.

Sample Input: $arr = \{1, 2, 3, 4, 5, 8\} \Rightarrow$ *Output:* Even: 3, Odd: 3

Q27. Difference Array. Given an array, create a new array where the value at index i is the absolute difference between `arr[i]` and `arr[i+1]`. For the last element, it should be the absolute difference between the last and the first element.

Constraints: $2 \leq N \leq 10^5$, $-10^9 \leq arr[i] \leq 10^9$.

Sample Input: `arr = {5, 9, 2}` $\Rightarrow$ *Output:* `{4, 7, 3}`

Explanation: $|5 - 9| = 4$, $|9 - 2| = 7$, $|2 - 5| = 3$.

Q28. Absolute Difference of Sums. Calculate the sum of elements at **even** indices and the sum of elements at **odd** indices in the array. Output the absolute difference between these two sums.

Constraints: $1 \leq N \leq 10^5$, $-10^4 \leq arr[i] \leq 10^4$.

Sample Input: `arr = {10, 5, 20, 15}` $\Rightarrow$ *Output:* 10

Explanation: Even indices sum = $10 + 20 = 30$. Odd indices sum = $5 + 15 = 20$. Difference = $|30 - 20| = 10$.

Q29. Minimum Absolute Difference. Find the minimum absolute difference between any two **adjacent** elements in the given array.

Constraints: $2 \leq N \leq 10^5$, $-10^9 \leq arr[i] \leq 10^9$.

Sample Input: `arr = {10, 20, 15, 30, 28}` $\Rightarrow$ *Output:* 2

Explanation: Differences: $|10 - 20| = 10$, $|20 - 15| = 5$, $|15 - 30| = 15$, $|30 - 28| = 2$. The minimum is 2.

Q30. Max Consecutive Ones. Given a binary array (containing only 0s and 1s), find the maximum number of consecutive 1s in the array.

Constraints: $1 \leq N \leq 10^5$, $arr[i] \in \{0, 1\}$.

Sample Input: `arr = {1, 1, 0, 1, 1, 1}` $\Rightarrow$ *Output:* 3

Explanation: The first two digits or the last three digits are consecutive 1s. The maximum is 3.